Speech Spoofing Detection Using Deep Neural Networks

Experimental Report on the ASVspoof 2019 LA Dataset

11 successful experiments out of 13 planned combinations:
4 feature extractors × 3 architectures + RawNet2

June 29, 2026

Contents

1 Introduction **2**

1.1 Background . . . 2

1.1.1 The ASVspoof Challenge Series . . . 3

1.2 Evaluation Metrics . . . 3

1.3 Experimental Objectives . . . 4

2 Related Work **6**

2.1 Traditional Approaches . . . 6

2.2 Deep Learning Approaches . . . 6

2.3 Feature Fusion and Ensemble Methods . . . 7

2.4 Positioning of This Work . . . 7

3 Data and Preprocessing **8**

3.1 The ASVspoof 2019 LA Dataset . . . 8

3.1.1 Attack Algorithms . . . 8

3.2 Preprocessing Pipeline . . . 9

4 Feature Extraction **11**

4.1 MFCC — Mel-Frequency Cepstral Coefficients . . . 11

4.1.1 The MFCC Extraction Pipeline . . . 11

4.1.2 Limitations of MFCC for Spoofing Detection . . . 14

4.2 LFCC — Linear Frequency Cepstral Coefficients . . . 14

4.2.1 Linear Filterbank Construction . . . 14

4.2.2 Why LFCC Works Better for Spoofing Detection . . . 15

4.3 CQCC — Constant-Q Cepstral Coefficients . . . 15

4.3.1 The Constant-Q Transform . . . 15

4.3.2 Time-Frequency Resolution Tradeoff . . . 16

4.3.3 From CQT to CQCC . . . 16

4.3.4 CQCC for Spoofing Detection . . . 17

4.4 SincConv — Learnable Sinc Filterbank . . . 17

4.4.1 Theoretical Foundation: Ideal Filters and the Sinc Function . . . 18

4.4.2 Windowing for Practical Implementation . . . 18
4.4.3 Parameterization and Learning . . . 18
4.4.4 Initialization . . . 19
4.4.5 Advantages and Limitations . . . 19
4.4.6 Comparison of Feature Extractors . . . 20

5 Neural Network Architectures 21
5.1 CNN — Baseline Convolutional Network . . . 21
5.1.1 The Convolution Operation . . . 21
5.1.2 Batch Normalization . . . 22
5.1.3 Max Pooling and Global Average Pooling . . . 22
5.2 Res2Net — Multi-Scale Residual Network . . . 23
5.2.1 Residual Learning . . . 23
5.2.2 Multi-Scale Feature Aggregation (Res2Net) . . . 23
5.2.3 Squeeze-and-Excitation (SE) Block . . . 24
5.3 LCNN — Light CNN with Max-Feature-Map . . . 25
5.3.1 Understanding MFM . . . 25
5.3.2 Why LCNN Works for Spoofing Detection . . . 26
5.4 RawNet2 — End-to-End Raw Waveform Network . . . 26
5.4.1 End-to-End Architecture Design . . . 26

6 Training and Evaluation 28
6.1 Training Configuration . . . 28
6.1.1 Adam Optimizer . . . 28
6.1.2 Binary Cross-Entropy Loss . . . 29
6.1.3 Callbacks and Training Strategy . . . 29
6.2 Data Generator . . . 29
6.3 EER Computation . . . 30
6.4 Simplified min-tDCF Computation . . . 30

7 Results 31
7.1 Summary . . . 31
7.2 Comparison Table . . . 31
7.3 Best System . . . 32
7.4 Comparison Chart . . . 32
7.5 Training Curves . . . 32

8 Discussion and Conclusion **35**
8.1 Feature Analysis . . . 35
8.2 Architecture Analysis . . . 36
8.3 The Generalization Gap . . . 36
8.4 Conclusion . . . 37

A Mathematical Foundations **39**
A.1 The Discrete Fourier Transform (DFT) . . . 39
A.1.1 Properties of the DFT . . . 39
A.1.2 The Fast Fourier Transform (FFT) . . . 40
A.2 The Discrete Cosine Transform (DCT) . . . 40
A.2.1 Why DCT for Cepstral Coefficients? . . . 40
A.3 The Mel Scale . . . 41
A.3.1 Psychoacoustic Basis . . . 41
A.4 The Sinc Function and Ideal Filters . . . 42
A.4.1 Properties . . . 42
A.4.2 Bandpass Construction . . . 42
A.5 Window Functions . . . 43
A.5.1 Rectangular Window . . . 43
A.5.2 Hann Window . . . 43
A.5.3 Hamming Window . . . 43
A.5.4 Tradeoff . . . 43
A.6 The Convolution Theorem . . . 44

1 Introduction

1.1 Background

Automatic Speaker Verification (ASV) systems authenticate a person's identity based on their voice. These systems have become widely deployed in banking, smart assistants, and access control. However, they are vulnerable to **spoofing attacks**—deliberate attempts to fool the system into accepting an impostor as a legitimate speaker.

The problem of spoofing is not merely theoretical. With the rapid advancement of Text-to-Speech (TTS) synthesis and Voice Conversion (VC) technologies, it has become increasingly easy to generate highly convincing fake speech. A spoofing **countermeasure** (CM) system is therefore a critical front-end that decides whether a given utterance is *bonafide* (genuine human speech) or *spoofed* (machine-generated or voice-converted), *before* the ASV system ever sees it.

Definition 1.1 ▷ Speech Spoofing

Speech spoofing refers to the production or transformation of speech signals with the intent to deceive an ASV system. The ASVspoof challenge categorizes attacks into three access types:

- **Logical Access (LA):** The attacker feeds a synthesized or converted waveform directly into the system, bypassing the microphone. Attack methods include TTS systems (neural vocoders, WaveNet, Tacotron) and VC systems (spectral mapping, phonetic posteriorgram-based conversion).
- **Physical Access (PA):** The attacker plays a recorded or synthesized utterance through a loudspeaker in front of the microphone (replay attack).
- **Deepfake (DF):** Introduced in ASVspoof 2021, this track evaluates robustness to compressed and codec-degraded deepfake audio, simulating real-world distribution channels (e.g., phone calls, messaging apps).

This report focuses exclusively on the **Logical Access (LA)** scenario.

1.1.1 The ASVspoof Challenge Series

The ASVspoof challenge has been the primary benchmark driving research in spoofing countermeasures:

- **ASVspoof 2015:** First edition; LA-only with relatively simple TTS/VC attacks. Baseline systems used GMM classifiers with MFCC and CQCC features.
- **ASVspoof 2017:** Focused on replay (PA) attacks with real recordings.
- **ASVspoof 2019:** Major expansion with both LA and PA tracks, 19 attack algorithms in the LA evaluation set (including neural vocoders), and the introduction of the tandem detection cost function (t-DCF) as the primary metric. This is the dataset used in our experiments.
- **ASVspoof 2021:** Added the DF track and evaluated robustness to telephony codecs and transmission channels.
- **ASVspoof 5 (2024):** Shifted toward open conditions with diverse attack types and a focus on generalization.

1.2 Evaluation Metrics

Two complementary metrics are used throughout this report.

Definition 1.2 ▷ Equal Error Rate (EER)

The **Equal Error Rate** is the operating point where the **False Acceptance Rate** (FAR) equals the **False Rejection Rate** (FRR). It provides a single-number summary of a binary classifier's discriminative power.

Given a set of scores $\{s_i\}$ and labels $\{y_i \in \{0, 1\}\}$, the FAR and FRR at a decision threshold τ are:

$$\text{FAR}(\tau) = \frac{|\{i : s_i \geq \tau \ \wedge \ y_i = 1\}|}{|\{i : y_i = 1\}|} \tag{1.1}$$

$$\text{FRR}(\tau) = \frac{|\{i : s_i < \tau \ \wedge \ y_i = 0\}|}{|\{i : y_i = 0\}|} \tag{1.2}$$

The EER is found at τ^* where $\text{FAR}(\tau^*) = \text{FRR}(\tau^*)$. In practice, this is computed by finding the threshold that minimizes $|\text{FAR} - \text{FRR}|$ on the ROC curve. **Lower EER is better**; a perfect system has EER $= 0\%$.

Intuition: EER answers the question "at what error rate can we simultaneously keep both types of mistakes equal?" It is threshold-independent, making it ideal for comparing systems without committing to a specific operating point.

Definition 1.3 ▷ Minimum Tandem Detection Cost Function (min-tDCF)

The **tandem DCF** is the official primary metric of the ASVspoof 2019 challenge. Unlike EER, it accounts for the *asymmetric costs* of different error types and incorporates the prior probability of spoofing attacks.
The general t-DCF is defined as:

$$t\text{-DCF}(\tau) = C_{\text{miss}} \cdot P_{\text{spoof}} \cdot P_{\text{miss}}(\tau) + C_{\text{fa}} \cdot (1 - P_{\text{spoof}}) \cdot P_{\text{fa}}(\tau) \tag{1.3}$$

where:

- C_{miss}: cost of rejecting a bonafide utterance (miss)
- C_{fa}: cost of accepting a spoofed utterance (false alarm)
- P_{spoof}: prior probability that an utterance is spoofed
- $P_{\text{miss}}(\tau)$ and $P_{\text{fa}}(\tau)$: miss and false alarm rates at threshold τ

The **minimum t-DCF** is obtained by optimizing over all possible thresholds:

$$\min\text{-}t\text{DCF} = \min_{\tau} \; t\text{-DCF}(\tau) \tag{1.4}$$

In our simplified CM-only evaluation, we normalize by the cost of the trivial "accept-all" system ($C_{\text{fa}} \cdot (1 - P_{\text{spoof}})$) so that $\min\text{-}t\text{DCF} \in [0, 1]$. We use $P_{\text{spoof}} = 0.05$, $C_{\text{miss}} = 1$, and $C_{\text{fa}} = 10$.

1.3 Experimental Objectives

The primary goal of this work is a **systematic comparison** of feature extraction methods and neural network architectures for speech spoofing detection. We design a grid of 13 experiments (4 feature extractors × 3 architectures + RawNet2) and evaluate each on the ASVspoof 2019 LA dataset under identical training conditions. Table 1.1 summarizes all planned combinations.

Table 1.1: Experimental combinations

#	**Feature**	**Model**	**Description**
1	MFCC	CNN	Mel-frequency cepstral coefficients + CNN
2	MFCC	Res2Net	Mel-frequency cepstral coefficients + Res2Net
3	MFCC	LCNN	Mel-frequency cepstral coefficients + LCNN
4	LFCC	CNN	Linear-frequency cepstral coefficients + CNN
5	LFCC	Res2Net	Linear-frequency cepstral coefficients + Res2Net
6	LFCC	LCNN	Linear-frequency cepstral coefficients + LCNN
7	CQCC	CNN	Constant-Q cepstral coefficients + CNN
8	CQCC	Res2Net	Constant-Q cepstral coefficients + Res2Net
9	CQCC	LCNN	Constant-Q cepstral coefficients + LCNN
10	SincConv	CNN	Learnable filterbank + CNN
11	SincConv	Res2Net	Learnable filterbank + Res2Net
12	SincConv	LCNN	Learnable filterbank + LCNN
13	Raw	RawNet2	Direct raw waveform processing

2 Related Work

2.1 Traditional Approaches

Early spoofing countermeasures relied on handcrafted features fed into statistical classifiers. The ASVspoof 2015 baseline used a **Gaussian Mixture Model (GMM)** back-end with MFCC or CQCC front-ends. These systems achieved reasonable performance on known attack types but struggled to generalize to unseen synthesis methods.

Key developments in the traditional era include:

- **CQCC + GMM** (Todisco et al., 2017): The official ASVspoof 2017 baseline; CQCC's variable time-frequency resolution proved effective for capturing artifacts across the frequency spectrum.
- **LFCC + GMM** (Sahidullah et al., 2015): Demonstrated that linear-frequency cepstral coefficients outperform MFCC for spoofing detection because they preserve high-frequency information where synthesis artifacts are most prominent.
- **Phase-based features**: Modified Group Delay (MGD) and instantaneous frequency features were explored to capture unnatural phase patterns in synthetic speech.

2.2 Deep Learning Approaches

The shift to deep learning brought significant improvements. Notable architectures include:

Key Deep Learning Systems

- **LCNN** (Wu et al., 2018): Light CNN with Max-Feature-Map activation; became a widely-used baseline due to its strong performance with relatively few parameters.
- **Res2Net + SE** (Li et al., 2021): Multi-scale residual networks with squeeze-and-excitation attention; achieved competitive results on ASVspoof 2019 LA.
- **RawNet2** (Tak et al., 2021): End-to-end system processing raw waveforms via a SincConv front-end and residual blocks with GRU aggregation; official ASVspoof

2021 baseline.

- **AASIST** (Jung et al., 2022): Audio Anti-Spoofing using Integrated Spectro-Temporal graph attention networks; models spectral and temporal dependencies as heterogeneous graphs with a Max Graph Operation for fusion. Achieved EER as low as 0.83% on ASVspoof 2019 LA evaluation.
- **SSL-based systems** (wav2vec 2.0, WavLM, HuBERT): Self-supervised speech representations fine-tuned for spoofing detection have shown the best generalization to unseen attacks, with EER below 1% on ASVspoof 2019 LA.

2.3 Feature Fusion and Ensemble Methods

Score-level and embedding-level fusion of multiple systems has consistently outperformed single-system approaches in ASVspoof challenges. Common strategies include:

- Averaging or weighted combination of detection scores from different feature/model pairs.
- Concatenation of embeddings from heterogeneous front-ends before a shared classifier.
- Bosaris-style logistic regression calibration and fusion.

2.4 Positioning of This Work

Our work does not aim to achieve state-of-the-art performance. Instead, it provides a controlled, **apples-to-apples comparison** across four classical feature extractors (MFCC, LFCC, CQCC, SincConv) and three neural architectures (CNN, Res2Net, LCNN), plus RawNet2 as an end-to-end baseline. All experiments share identical training hyperparameters, preprocessing, and evaluation protocols. This design isolates the effect of each feature–architecture combination, providing insights that are often obscured in challenge submissions where every team uses different settings.

3 Data and Preprocessing

3.1 The ASVspoof 2019 LA Dataset

The ASVspoof 2019 Logical Access dataset is a standard benchmark for evaluating spoofing countermeasures. All utterances are single-channel, 16-bit PCM audio sampled at 16 kHz. The dataset is partitioned into three disjoint subsets with **non-overlapping attack algorithms**—a critical design choice that tests generalization to unseen spoofing methods.

Dataset Specifications

- **Sample rate:** 16 000 Hz (narrowband telephony quality)
- **Fixed duration after cropping:** 4 seconds (64 000 samples)
- **Protocols:** train / dev / eval
- **Labels:** `bonafide` (genuine, label 0) and `spoof` (fake, label 1)
- **Attack types:** A01–A06 (train/dev), A07–A19 (eval only)

Table 3.1: Sample distribution in the ASVspoof 2019 LA dataset

Partition	Bonafide	Spoof	Total
Train	2 580	22 800	25 380
Dev	2 548	22 296	24 844
Eval	7 355	63 882	71 237

Note

The dataset is **highly imbalanced**: spoofed utterances outnumber bonafide ones by roughly 9:1. Furthermore, the evaluation set contains 13 attack algorithms (A07–A19) that are *entirely absent* from training and development. This deliberate mismatch is the primary source of the generalization gap observed across all experiments.

3.1.1 Attack Algorithms

The train and dev partitions share the same six attack algorithms:

- **A01–A04:** TTS systems based on neural and statistical parametric synthesis (e.g., waveform concatenation, WaveNet-based vocoders).
- **A05–A06:** Voice conversion systems using spectral mapping and frequency warping techniques.

The evaluation set introduces 13 additional algorithms (A07–A19), including more advanced neural vocoders (WaveRNN, WaveGlow), end-to-end TTS systems, and sophisticated VC methods. This ensures that any system achieving low eval EER has genuinely learned to detect *spoofing artifacts in general*, not just memorized the specific attack signatures seen during training.

3.2 Preprocessing Pipeline

All audio files pass through a mandatory preprocessing pipeline before feature extraction.

Method 3.1 ▷ Mandatory Preprocessing

Applied to **every** utterance in every experiment:

1. **Loading:** Read the FLAC file using `librosa.load` with `sr=16000` to ensure consistent sampling rate.
2. **Amplitude normalization:** Scale to unit peak amplitude:

$$x[n] \leftarrow \frac{x[n]}{\max_n |x[n]| + \epsilon} \quad \text{where} \quad \epsilon = 10^{-8} \tag{3.1}$$

   This prevents the model from using loudness as a spurious discriminative feature.
3. **Fixed-length adjustment:** Crop or tile-and-crop to exactly 64 000 samples (4 seconds). During training, a random starting position is chosen; during evaluation, the center crop is used.

Method 3.2 ▷ Optional Preprocessing

Configurable flags that enable additional processing:

`USE_VAD` **Silence trimming** using energy-based Voice Activity Detection (`librosa.effects.trim` with `top_db=30`). Removes leading and trailing silence so the model focuses on speech content rather than dataset-specific silence patterns. If trimming would reduce the signal below 0.1 seconds, the original waveform is kept.

`USE_RAWBOOST` **Noise augmentation** applied with 50% probability during training.

Combines two noise types:

- *Additive Gaussian noise* at a random SNR between 10–40 dB.
- *Impulsive noise* at 0.1% of sample positions, simulating clicks and pops from recording artifacts.

This is inspired by the RawBoost augmentation strategy (Tak et al., 2022), which showed that even simple noise injection improves robustness.

`USE_REVERB` **Simple reverberation** applied with 30% probability during training. A single delayed copy of the signal is added with random decay factor (0.1–0.4) and random delay (400–2400 samples, i.e., 25–150 ms). This simulates room acoustics without requiring real Room Impulse Responses (RIRs).

`USE_DENOISING` **Spectral subtraction denoising.** Estimates the noise floor from the first 10 STFT frames and subtracts twice this estimate from the magnitude spectrum, with a floor at 10% of the original magnitude to avoid musical noise artifacts. Primarily exploratory—aggressive denoising risks removing spoofing artifacts along with the noise.

Note

In our experiments, `USE_VAD` and `USE_RAWBOOST` are **enabled**; `USE_REVERB` and `USE_DENOISING` are **disabled**.

4 Feature Extraction

Audio feature extraction transforms a raw waveform into a compact, informative representation that emphasizes characteristics relevant to the downstream task—in our case, distinguishing bonafide speech from spoofed speech. This chapter provides a detailed, step-by-step treatment of each feature extraction method used in our experiments.

4.1 MFCC — Mel-Frequency Cepstral Coefficients

MFCCs are the most widely used features in speech processing. Introduced by Davis and Mermelstein (1980), they compress the spectral envelope of speech into a small number of decorrelated coefficients that roughly correspond to the smooth shape of the vocal tract transfer function.

Definition 4.1 ▷ MFCC

Mel-Frequency Cepstral Coefficients are obtained by applying the Discrete Cosine Transform (DCT) to the logarithm of mel-scaled filterbank energies. They capture the spectral envelope while discarding fine harmonic structure.
Output shape in our experiments: $(N_{\text{frames}}, 60, 1)$.

4.1.1 The MFCC Extraction Pipeline

The MFCC computation proceeds through six sequential stages:

Method 4.1 ▷ MFCC Pipeline

1. **Framing:** The continuous waveform is divided into overlapping short-time frames. Each frame has length $N_w = 400$ samples (25 ms at 16 kHz) with a hop size of $H = 160$ samples (10 ms), giving 60% overlap. For a signal of length L, the number of frames is:

$$N_{\text{frames}} = \left\lfloor \frac{L - N_w}{H} \right\rfloor + 1 \tag{4.1}$$

With $L = 64\,000$ and $H = 160$: $N_{\text{frames}} = 400$.

2. **Windowing:** Each frame is multiplied element-wise by a window function to reduce spectral leakage from the abrupt truncation at frame boundaries. The Hann window is typically used:

$$w[n] = 0.5 - 0.5\cos\left(\frac{2\pi n}{N_w - 1}\right), \quad n = 0, 1, \ldots, N_w - 1 \tag{4.2}$$

The windowed frame is $\tilde{x}_m[n] = x_m[n] \cdot w[n]$, where x_m is the m-th frame.

3. **Short-Time Fourier Transform (STFT):** The DFT of each windowed frame computes the frequency-domain representation:

$$X_m[k] = \sum_{n=0}^{N_{\text{FFT}}-1} \tilde{x}_m[n]\, e^{-j2\pi kn/N_{\text{FFT}}}, \quad k = 0, 1, \ldots, N_{\text{FFT}} - 1 \tag{4.3}$$

We use $N_{\text{FFT}} = 512$. The **power spectrum** of each frame is:

$$P_m[k] = \frac{1}{N_{\text{FFT}}} |X_m[k]|^2 \tag{4.4}$$

Only the first $N_{\text{FFT}}/2 + 1 = 257$ bins are kept (the rest are conjugate symmetric for real-valued signals).

4. **Mel filterbank:** The power spectrum is passed through a bank of M triangular filters spaced according to the **mel scale**. The mel scale approximates the nonlinear frequency perception of the human auditory system:

$$m(f) = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right) \tag{4.5}$$

and its inverse:

$$f(m) = 700 \cdot \left(10^{m/2595} - 1\right) \tag{4.6}$$

The triangular filters are defined by center frequencies $\{c_i\}_{i=1}^{M}$ that are *uniformly spaced on the mel scale* (equivalently, logarithmically spaced on the Hz scale at

higher frequencies). Each filter $H_i[k]$ is:

$$H_i[k] = \begin{cases} \dfrac{f[k] - c_{i-1}}{c_i - c_{i-1}} & c_{i-1} \leq f[k] < c_i \\ \dfrac{c_{i+1} - f[k]}{c_{i+1} - c_i} & c_i \leq f[k] \leq c_{i+1} \\ 0 & \text{otherwise} \end{cases} \tag{4.7}$$

The filterbank energy for the m-th frame and i-th filter is:

$$E_m[i] = \sum_{k=0}^{N_{\text{FFT}}/2} H_i[k] \cdot P_m[k] \tag{4.8}$$

Why mel scaling? The human cochlea has higher frequency resolution at low frequencies (below ∼1 kHz) and progressively lower resolution above. The mel scale mimics this: filters are narrow and closely spaced at low frequencies (capturing fine pitch and formant details) and wide at high frequencies (where perceptual discrimination is coarser).

5. **Logarithmic compression:** The log of each filterbank energy is taken:

$$\hat{E}_m[i] = \log(E_m[i] + \epsilon) \tag{4.9}$$

This compresses the dynamic range (matching the ear's roughly logarithmic loudness perception) and converts the multiplicative effects of the source-filter model into additive effects in the log domain, which DCT can then separate.

6. **Discrete Cosine Transform (DCT):** Finally, the Type-II DCT is applied across the M log-filterbank energies to produce N_{MFCC} cepstral coefficients:

$$c_m[j] = \sum_{i=0}^{M-1} \hat{E}_m[i] \cdot \cos\left[\frac{\pi j(2i+1)}{2M}\right], \quad j = 0, 1, \ldots, N_{\text{MFCC}} - 1 \tag{4.10}$$

Why DCT? The mel filterbank energies are correlated because adjacent triangular filters overlap. The DCT decorrelates them (it is the optimal energy-compacting transform for first-order Markov processes), concentrating most of the spectral envelope information into the first few coefficients. This is why traditionally only 13–20 MFCCs are used, though we use $N_{\text{MFCC}} = 60$ to provide the neural network with richer input.

4.1.2 Limitations of MFCC for Spoofing Detection

While MFCCs excel at capturing the broad spectral shape relevant to speech recognition, they have a known weakness for spoofing detection: the mel-scale filterbank **progressively discards high-frequency information**. Synthesis artifacts from vocoders (e.g., buzzy harmonics, unnatural spectral tilt, aliasing) often manifest at frequencies above 4 kHz, where mel filters are wide and smear spectral detail. This motivates the use of LFCC.

Our parameters: window 400 samples, hop 160 samples, $N_{\text{FFT}} = 512$, $N_{\text{MFCC}} = 60$.

4.2 LFCC — Linear Frequency Cepstral Coefficients

Definition 4.2 ▷ LFCC

Linear Frequency Cepstral Coefficients follow the same pipeline as MFCC, but replace the mel-scaled filterbank with a **linearly-spaced filterbank**. This preserves equal frequency resolution across the entire spectrum, retaining high-frequency details that mel scaling would blur.
Output shape: $(N_{\text{frames}}, 60, 1)$.

4.2.1 Linear Filterbank Construction

Instead of spacing filter centers on the mel scale, LFCC places them at **uniform intervals on the linear Hz scale**:

$$c_i = \frac{i \cdot f_{\max}}{M+1}, \quad i = 1, 2, \ldots, M \tag{4.11}$$

where $f_{\max} = f_s/2 = 8000\,\text{Hz}$ and $M = 70$ filters.

Each filter is still triangular with the same shape as in Equation (4.7), but now the filters at high frequencies have the *same bandwidth* as those at low frequencies. This means:

- **More resolution above 4 kHz** compared to mel filterbanks, which use increasingly wide filters in this range.
- **Less resolution below 1 kHz** compared to mel, because linear spacing allocates fewer filters there.

4.2.2 Why LFCC Works Better for Spoofing Detection

Speech synthesis and voice conversion algorithms introduce characteristic artifacts in the spectral domain:

- **Vocoder artifacts:** Neural vocoders (WaveNet, WaveGlow, WaveRNN) often produce subtle buzzing, spectral dropoffs, or aliasing patterns at high frequencies that differ from natural glottal excitation.
- **Spectral envelope smoothing:** Many TTS systems use mel-spectrograms as intermediate representations, inherently smoothing high-frequency structure. The resulting speech may sound natural to humans but has measurably different high-frequency statistics.
- **Band-limited synthesis:** Some older vocoders only synthesize up to a certain frequency and fill higher bands with shaped noise, creating detectable boundaries.

By preserving **uniform resolution at all frequencies**, LFCC captures these artifacts far more effectively than MFCC. This is why LFCC has been a consistently strong feature for ASVspoof since Sahidullah et al. (2015) first demonstrated its advantage.

The rest of the pipeline (STFT → filterbank → log → DCT) is identical to MFCC. We use $M = 70$ linear filters and retain $N_{\text{LFCC}} = 60$ DCT coefficients.

4.3 CQCC — Constant-Q Cepstral Coefficients

Definition 4.3 ▷ CQCC

Constant-Q Cepstral Coefficients are derived from the Constant-Q Transform (CQT) rather than the STFT. The CQT provides logarithmic frequency resolution—high resolution at low frequencies and lower resolution at high frequencies—with a variable time-frequency tradeoff that better matches musical and speech signal structure.
Output shape: $(N_{\text{frames}}, 60, 1)$.

4.3.1 The Constant-Q Transform

The standard STFT has a **fixed** frequency resolution $\Delta f = f_s/N_{\text{FFT}}$ at every frequency bin, meaning the ratio $Q = f/\Delta f$ increases linearly with frequency. The CQT instead maintains

a **constant Q factor** across all bins, achieved by using different window lengths at different frequencies.

The CQT of a signal $x[n]$ at frequency bin k is:

$$X_{\text{CQ}}[k] = \frac{1}{N_k} \sum_{n=0}^{N_k-1} x[n] \cdot w_k[n] \cdot e^{-j2\pi Qn/N_k} \tag{4.12}$$

where N_k is the window length for bin k, defined so that each bin spans exactly Q oscillation cycles:

$$N_k = \frac{Q \cdot f_s}{f_k} \tag{4.13}$$

The center frequencies are geometrically (logarithmically) spaced:

$$f_k = f_{\min} \cdot 2^{k/B}, \quad k = 0, 1, \ldots, K-1 \tag{4.14}$$

where B is the number of bins per octave and K is the total number of bins. The Q factor is:

$$Q = \frac{1}{2^{1/B} - 1} \tag{4.15}$$

4.3.2 Time-Frequency Resolution Tradeoff

The CQT exhibits the *opposite* resolution tradeoff compared to STFT:

	Low frequencies	**High frequencies**
STFT	Fixed Δf, fixed Δt	Fixed Δf, fixed Δt
CQT	Fine Δf, coarse Δt	Coarse Δf, fine Δt

At low frequencies, the CQT uses long windows (high frequency resolution, needed to resolve closely-spaced pitch harmonics) but sacrifices temporal resolution. At high frequencies, it uses short windows (high temporal resolution, useful for capturing transients like consonant bursts) but has wider frequency bins. This adaptive behavior makes CQT particularly suited for analyzing signals with rich harmonic structure, like speech and music.

4.3.3 From CQT to CQCC

After computing the CQT magnitude, the CQCC extraction follows the same log-DCT pattern as MFCC and LFCC:

1. Compute $|X_{\text{CQ}}[k, m]|^2$ (power spectrum per frame m).
2. Take the log: $\log(|X_{\text{CQ}}[k, m]|^2 + \epsilon)$.
3. Apply DCT across the K frequency bins to produce N_{CQCC} coefficients.

However, there is a subtlety: because CQT bins are geometrically spaced, the raw CQT output is **not uniformly sampled in time**. This requires resampling to a uniform time grid before the DCT step (Todisco et al., 2017). In our implementation via `librosa.cqt`, this resampling is handled internally.

Our parameters: 96 bins total, 24 bins per octave ($B = 24$, so $Q \approx 34.1$), minimum frequency $f_{\text{min}} = C1 \approx 32.7\,\text{Hz}$, and $N_{\text{CQCC}} = 60$ DCT coefficients.

4.3.4 CQCC for Spoofing Detection

CQCC was the **official baseline feature** for ASVspoof 2017 and 2019. Its advantages for spoofing detection include:

- Fine frequency resolution at low frequencies captures subtle differences in pitch and harmonic structure between natural and synthetic excitation.
- The logarithmic frequency spacing naturally matches the approximately log-spaced harmonics of voiced speech.
- Variable-length windows adapt the analysis to the local signal characteristics.

Its main disadvantage is computational cost (CQT is more expensive than FFT) and the resampling step, which can introduce interpolation artifacts.

4.4 SincConv — Learnable Sinc Filterbank

Definition 4.4 ▷ SincConv1D

SincConv1D is a convolutional layer whose kernels are parameterized as **sinc-based bandpass filters**. Instead of learning arbitrary kernel weights, only the low and high cutoff frequencies (f_1, f_2) of each bandpass filter are learned. This dramatically reduces the number of parameters while enforcing a physically meaningful filterbank structure.
Input: raw waveform $(N_{\text{samples}}, 1)$.
Output: feature map $(N_{\text{samples}}, N_{\text{filters}})$.

4.4.1 Theoretical Foundation: Ideal Filters and the Sinc Function

An ideal lowpass filter with cutoff frequency f_c has a frequency response that is 1 for $|f| \leq f_c$ and 0 elsewhere. Its impulse response (the inverse Fourier transform of this rectangular frequency response) is the **sinc function**:

$$h_{\text{LP}}[n] = 2f_c \cdot \text{sinc}(2\pi f_c n) = 2f_c \cdot \frac{\sin(2\pi f_c n)}{2\pi f_c n} \tag{4.16}$$

An ideal **bandpass** filter passing frequencies between f_1 and f_2 can be constructed as the *difference* of two lowpass filters:

$$h_{\text{BP}}[n] = 2f_2 \,\text{sinc}(2\pi f_2 n) - 2f_1 \,\text{sinc}(2\pi f_1 n) \tag{4.17}$$

This is because in the frequency domain, subtraction of the two rectangular responses yields a rectangle from f_1 to f_2.

4.4.2 Windowing for Practical Implementation

The ideal sinc filter has infinite support (it extends to $\pm\infty$), so it must be truncated to a finite kernel of length K. Truncation introduces ripples in the frequency response (Gibbs phenomenon). A **Hamming window** is applied to smooth these ripples:

$$w[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{K-1}\right) \tag{4.18}$$

The final practical bandpass kernel is:

$$h[n] = [2f_2 \,\text{sinc}(2\pi f_2 n) - 2f_1 \,\text{sinc}(2\pi f_1 n)] \cdot w[n] \tag{4.19}$$

In our implementation, the kernel is normalized to unit peak amplitude after windowing.

4.4.3 Parameterization and Learning

Each of the N_{filters} sinc filters is defined by exactly **two learnable parameters**: $f_1^{(i)}$ and $f_2^{(i)}$. To ensure valid bandpass behavior, we enforce:

$$f_1^{(i)} = |f_1^{(i)}|, \quad f_2^{(i)} = |f_2^{(i)}| + f_1^{(i)} + \delta \tag{4.20}$$

where $\delta = 10^{-5}$ prevents degenerate zero-bandwidth filters.

During backpropagation, gradients flow through the sinc function to update f_1 and f_2. The key derivative is:

$$\frac{\partial h_{\text{LP}}[n]}{\partial f_c} = \begin{cases} 2\,\text{sinc}(2\pi f_c n) + 2f_c \cdot \frac{2\pi n \cos(2\pi f_c n) - \sin(2\pi f_c n)}{(2\pi f_c n)^2} & n \neq 0 \\ 2 & n = 0 \end{cases} \tag{4.21}$$

This means the network can learn to **shift and widen/narrow each filter's passband** during training, adapting the filterbank to the specific task.

4.4.4 Initialization

Filters are initialized with cutoff frequencies on the **mel scale**, identical to a standard mel filterbank. This gives the network a good starting point that is already useful for speech processing, while allowing it to deviate if the task demands it.

$$f_1^{(i)} = \text{mel}^{-1}(\text{linspace}(m(30), m(f_s/2 - 50), N+2)_i)\,/f_s \tag{4.22}$$

$$f_2^{(i)} = \text{mel}^{-1}(\text{linspace}(m(30), m(f_s/2 - 50), N+2)_{i+2})\,/f_s \tag{4.23}$$

4.4.5 Advantages and Limitations

- **Advantages:** Far fewer parameters than standard Conv1D (only $2N_{\text{filters}}$ vs. $N_{\text{filters}} \times K$); enforces meaningful filterbank structure; can discover task-specific frequency bands that handcrafted features miss.
- **Limitations:** Constrained to bandpass shapes (cannot learn arbitrary spectral patterns); requires more training epochs to optimize filter boundaries; gradient magnitude varies significantly across the kernel, potentially causing slow convergence.

Note

When SincConv is used as a standalone front-end (experiments 10–12), its output is downsampled via average pooling to produce a 2D feature map of shape $(N_{\text{frames}}, N_{\text{filters}}, 1)$, which then feeds into CNN, Res2Net, or LCNN. In RawNet2 (experiment 13), SincConv is the first layer of the full architecture.

4.4.6 Comparison of Feature Extractors

Table 4.1 summarizes the key characteristics of all four feature extractors.

Table 4.1: Comparison of feature extraction methods

Feature	**Freq. scale**	**Learnable?**	**High-freq**	**Params**	**Key strength**
MFCC	Mel (log)	No	Low	0	Perceptual relevance
LFCC	Linear	No	High	0	Uniform resolution
CQCC	Log (CQT)	No	Med	0	Adaptive time-freq
SincConv	Learnable	Yes	Task-dep.	$2N$	Task-adapted filters

5 Neural Network Architectures

This chapter details the four neural network architectures used as classifiers (or, in the case of RawNet2, as a complete end-to-end system). We provide the mathematical formulations of key operations and explain the intuition behind architectural choices.

5.1 CNN — Baseline Convolutional Network

Method 5.1 ▷ CNN Architecture

Four sequential blocks, each containing:

1. Two Conv2D layers with filters $\{32, 64, 128, 256\}$, kernel size 3×3
2. Batch Normalization + ReLU after each convolution
3. MaxPool2D(2×2) + Dropout(0.1)

Head: GlobalAveragePooling2D $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dropout(0.3) $\rightarrow$ Dense(1, Sigmoid)

5.1.1 The Convolution Operation

A 2D convolution slides a small learnable kernel $\boldsymbol{W} \in \mathbb{R}^{k_h \times k_w}$ over the input feature map $\boldsymbol{X}$ and computes a weighted sum at each position:

$$Y[i,j] = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} W[m,n] \cdot X[i+m, j+n] + b \tag{5.1}$$

where b is a learnable bias. With "same" padding, the output has the same spatial dimensions as the input. The kernel size 3×3 is a standard choice that balances local receptive field size with parameter efficiency.

Receptive field: After L convolutional layers (each 3×3) without pooling, each output neuron "sees" a $(2L+1) \times (2L+1)$ region of the input. With MaxPool2D(2) between blocks, the effective receptive field grows much faster, allowing deeper layers to capture progressively larger-scale patterns in the spectrogram.

5.1.2 Batch Normalization

Batch Normalization (Ioffe & Szegedy, 2015) normalizes activations within each mini-batch to stabilize and accelerate training:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \tag{5.2}$$

$$y_i = \gamma \hat{x}_i + \beta \tag{5.3}$$

where $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are the mini-batch mean and variance, and γ, β are learnable scale and shift parameters. This reduces internal covariate shift—the phenomenon where the distribution of layer inputs changes as the parameters of preceding layers change during training.

5.1.3 Max Pooling and Global Average Pooling

MaxPool2D(2×2) with stride 2 reduces spatial dimensions by half, selecting the maximum value in each 2×2 window:

$$Y[i, j] = \max_{0 \le m,n < 2} X[2i + m, 2j + n] \tag{5.4}$$

This provides translation invariance and progressive spatial reduction. After four pooling operations, spatial dimensions shrink by $16\times$ (e.g., $400 \times 60 \to 25 \times 3$).

Global Average Pooling (GAP) computes the spatial mean of each feature map channel, producing a single value per channel regardless of input spatial size:

$$z_c = \frac{1}{H \times W} \sum_{i=1}^{H} \sum_{j=1}^{W} X_c[i, j] \tag{5.5}$$

GAP acts as a structural regularizer: it has no learnable parameters and forces each feature map to represent a single concept, reducing overfitting compared to flattening followed by a dense layer.

5.2 Res2Net — Multi-Scale Residual Network

Method 5.2 ▷ Res2Net Architecture

Initial Conv2D(7×7) → BatchNorm → ReLU → MaxPool2D(3×3), followed by three Res2Net blocks with $\{64, 128, 256\}$ filters. Each block includes a Squeeze-and-Excitation (SE) module.
Head: GlobalAveragePooling2D → Dense(128, ReLU) → Dropout(0.3) → Dense(1, Sigmoid)

5.2.1 Residual Learning

The fundamental principle behind ResNet (He et al., 2016) is **residual learning**: instead of learning a mapping $\mathcal{H}(\boldsymbol{x})$ directly, the network learns the residual $\mathcal{F}(\boldsymbol{x}) = \mathcal{H}(\boldsymbol{x}) - \boldsymbol{x}$, and the output is:

$$\boldsymbol{y} = \mathcal{F}(\boldsymbol{x}) + \boldsymbol{x} \tag{5.6}$$

The identity shortcut $\boldsymbol{x}$ allows gradients to flow directly through the network, mitigating the vanishing gradient problem in deep architectures. If the optimal transformation is close to identity (common in deep networks), it is easier for the network to push $\mathcal{F}(\boldsymbol{x})$ toward zero than to learn $\mathcal{H}(\boldsymbol{x}) \approx \boldsymbol{x}$.

5.2.2 Multi-Scale Feature Aggregation (Res2Net)

Res2Net (Gao et al., 2021) extends standard residual blocks by capturing **multi-scale features** within a single block. The key idea is to split the intermediate feature maps into s groups (called "scales") and process them in a hierarchical cascade:

1. **Split:** After the first 1×1 convolution, the channel dimension is divided into s equal groups: $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_s$, each with C/s channels.
2. **Hierarchical processing:** Each group (except the first) is processed by a 3×3 convolution, but its input includes the output of the *previous* group:

$$\boldsymbol{y}_i = \begin{cases} \boldsymbol{x}_i & i = 1 \\ K_i(\boldsymbol{x}_i + \boldsymbol{y}_{i-1}) & i = 2, 3, \ldots, s \end{cases} \tag{5.7}$$

where K_i denotes the 3×3 conv + BN + ReLU for group i.

3. **Concatenation:** All group outputs are concatenated along the channel axis, followed by a final 1×1 convolution to restore the original channel count.

This cascade structure means that group i has an **effective receptive field** that is i times larger than a single 3×3 convolution. Group 1 captures local patterns (3×3), group 2 captures slightly larger patterns (effectively 5×5), and so on up to group s (effectively $(2s+1) \times (2s+1)$). This multi-scale representation is particularly valuable for spectrogram inputs, where spoofing artifacts may appear at different time-frequency scales.

In our implementation, $s = 4$ (the standard Res2Net configuration).

5.2.3 Squeeze-and-Excitation (SE) Block

The SE block (Hu et al., 2018) performs **channel-wise attention**—it learns to emphasize informative feature channels and suppress less useful ones:

1. **Squeeze:** Global Average Pooling compresses each channel into a scalar:

$$z_c = \frac{1}{H \times W} \sum_{i,j} X_c[i, j] \tag{5.8}$$

2. **Excitation:** Two fully-connected layers with a bottleneck produce channel weights:

$$\boldsymbol{s} = \sigma(\boldsymbol{W}_2 \cdot \mathrm{ReLU}(\boldsymbol{W}_1 \cdot \boldsymbol{z})) \tag{5.9}$$

   where $\boldsymbol{W}_1 \in \mathbb{R}^{(C/r) \times C}$ and $\boldsymbol{W}_2 \in \mathbb{R}^{C \times (C/r)}$ with reduction ratio $r = 8$.
3. **Scale:** The input feature map is re-weighted channel-wise:

$$\tilde{X}_c = s_c \cdot X_c \tag{5.10}$$

The sigmoid σ produces weights in $[0, 1]$, so each channel can be amplified (weight near 1) or suppressed (weight near 0) based on the global context captured by GAP.

5.3 LCNN — Light CNN with Max-Feature-Map

Definition 5.1 ▷ Max-Feature-Map (MFM) Activation

The **Max-Feature-Map** operation splits a layer's output channels into two equal halves and takes the element-wise maximum:

$$\text{MFM}(\boldsymbol{X}) = \max(\boldsymbol{X}_{:,:,1:C/2},\ \boldsymbol{X}_{:,:,C/2+1:C}) \tag{5.11}$$

This simultaneously performs **nonlinear activation** (like ReLU) and **channel reduction** (halving the channel count), acting as a form of competitive feature selection.

Method 5.3 ▷ LCNN Architecture

Five blocks of Conv2D + MFM + BatchNorm, with MaxPool2D after blocks 1, 3, and 5. Channel progression: $64 \to 64 \to 128 \to 128 \to 128$ (before MFM halving).
Head: GlobalAveragePooling2D $\to$ MFM(Dense(256)) $\to$ Dense(128, ReLU) $\to$ Dropout(0.3) $\to$ Dense(1, Sigmoid)

5.3.1 Understanding MFM

MFM can be understood from multiple perspectives:

1. As competitive feature selection: Each pair of channels "competes," and only the larger activation survives. This forces the network to learn complementary feature detectors: if channel a fires strongly for pattern A and channel b fires strongly for pattern B, MFM selects whichever pattern is more prominent at each spatial location.

2. Comparison with ReLU: ReLU applies a fixed threshold (zero) uniformly:

$$\text{ReLU}(x) = \max(x, 0) \tag{5.12}$$

MFM applies an **adaptive, input-dependent threshold**—the activation of the paired channel serves as the threshold. This makes MFM strictly more expressive than ReLU: if one channel always outputs zero, MFM degenerates to ReLU. In practice, MFM learns more nuanced gating behavior.

3. As dimensionality reduction: Each MFM layer halves the channel count, so a Conv2D with $2C$ output channels followed by MFM produces C channels. This built-in compression

keeps the model lightweight.

5.3.2 Why LCNN Works for Spoofing Detection

The competitive selection property of MFM is particularly well-suited for spoofing detection, where the discriminative signal is subtle. Rather than simply thresholding activations at zero (ReLU), MFM allows the network to dynamically select the most informative features at each time-frequency location. This implicit feature selection mechanism helps LCNN focus on the fine-grained spectral differences between bonafide and spoofed speech.

5.4 RawNet2 — End-to-End Raw Waveform Network

Method 5.4 ▷ RawNet2 Architecture

The only model that operates directly on the raw audio waveform:

1. SincConv1D(128 filters, kernel size 1025) + BatchNorm + MaxPool1D(3)
2. Two 1D Residual Blocks with 128 channels + MaxPool1D(3)
3. Two 1D Residual Blocks with 256 channels + MaxPool1D(3)
4. Two 1D Residual Blocks with 512 channels + MaxPool1D(3)
5. GRU(1024) → Dense(1024, ReLU) → Dropout(0.3) → Dense(1, Sigmoid)

5.4.1 End-to-End Architecture Design

RawNet2 (Tak et al., 2021) eliminates the separate feature extraction step by learning directly from the waveform. The architecture consists of three stages:

1. Learnable front-end (SincConv): The SincConv layer (Section 4.4) acts as a learnable filterbank, replacing handcrafted features like MFCC or LFCC. With 128 filters and kernel size 1025 (≈64 ms at 16 kHz), it covers a wide temporal context for each filter output.

2. 1D Residual backbone: Six residual blocks with progressive channel expansion (128 → 256 → 512) extract hierarchical representations from the filterbank output. Each 1D residual block follows the same structure as standard ResNet blocks but uses 1D convolutions:

$$\boldsymbol{y} = \text{LeakyReLU}_{0.3}(\text{BN}(\text{Conv1D}(\text{LeakyReLU}_{0.3}(\text{BN}(\text{Conv1D}(\boldsymbol{x}))))) + \boldsymbol{x}_{\text{proj}}) \tag{5.13}$$

where $\boldsymbol{x}_{\text{proj}}$ is the shortcut connection, projected via 1×1 convolution if the channel dimensions differ. LeakyReLU with slope 0.3 is used instead of ReLU to prevent dead neurons in

the 1D domain.

MaxPool1D(3) between groups of two blocks progressively reduces the temporal dimension: $64000 \to 21333 \to 7111 \to 2370 \to 790$.

3. Temporal aggregation (GRU): A **Gated Recurrent Unit** (Cho et al., 2014) aggregates the temporal sequence into a fixed-length representation. The GRU processes the sequence step-by-step, maintaining a hidden state $\boldsymbol{h}_t$ that summarizes all information up to time t:

$$\boldsymbol{z}_t = \sigma(\boldsymbol{W}_z \boldsymbol{x}_t + \boldsymbol{U}_z \boldsymbol{h}_{t-1} + \boldsymbol{b}_z) \quad \text{(update gate)} \tag{5.14}$$

$$\boldsymbol{r}_t = \sigma(\boldsymbol{W}_r \boldsymbol{x}_t + \boldsymbol{U}_r \boldsymbol{h}_{t-1} + \boldsymbol{b}_r) \quad \text{(reset gate)} \tag{5.15}$$

$$\tilde{\boldsymbol{h}}_t = \tanh(\boldsymbol{W}_h \boldsymbol{x}_t + \boldsymbol{U}_h (\boldsymbol{r}_t \odot \boldsymbol{h}_{t-1}) + \boldsymbol{b}_h) \quad \text{(candidate)} \tag{5.16}$$

$$\boldsymbol{h}_t = (1 - \boldsymbol{z}_t) \odot \boldsymbol{h}_{t-1} + \boldsymbol{z}_t \odot \tilde{\boldsymbol{h}}_t \quad \text{(output)} \tag{5.17}$$

where σ is the sigmoid function and $\odot$ denotes element-wise multiplication.

The **update gate** $\boldsymbol{z}_t$ controls how much of the previous state to retain vs. how much to update with new information. The **reset gate** $\boldsymbol{r}_t$ controls how much of the previous state is used to compute the candidate. The final hidden state $\boldsymbol{h}_T$ (output of the last time step) serves as the utterance-level embedding.

Note

RawNet2 uses the SincConv layer as part of its architecture, not as a separate preprocessing step. This is different from experiments 10–12, where SincConv output is reshaped into a 2D feature map and fed into CNN/Res2Net/LCNN.

6 Training and Evaluation

6.1 Training Configuration

All experiments share identical training hyperparameters to ensure a fair comparison.

Training Configuration

- **Optimizer:** Adam (Kingma & Ba, 2015) with initial learning rate $\eta = 10^{-4}$
- **Loss function:** Binary cross-entropy
- **Batch size:** 32
- **Maximum epochs:** 30
- **Early stopping:** Patience of 7 epochs, monitoring `val_loss`
- **Learning rate reduction:** Factor 0.5 after 3 epochs without improvement (minimum 10^{-6})
- **Best weights:** Checkpoint saved based on lowest `val_loss`

6.1.1 Adam Optimizer

Adam combines the benefits of two earlier methods: AdaGrad (which adapts learning rates per-parameter based on historical gradient magnitudes) and RMSProp (which uses an exponentially decaying average of squared gradients). The update rule maintains two moment estimates:

$$\boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1 - \beta_1)\boldsymbol{g}_t \qquad \text{(first moment — mean)} \tag{6.1}$$

$$\boldsymbol{v}_t = \beta_2 \boldsymbol{v}_{t-1} + (1 - \beta_2)\boldsymbol{g}_t^2 \qquad \text{(second moment — variance)} \tag{6.2}$$

Bias-corrected estimates account for initialization at zero:

$$\hat{\boldsymbol{m}}_t = \frac{\boldsymbol{m}_t}{1 - \beta_1^t}, \qquad \hat{\boldsymbol{v}}_t = \frac{\boldsymbol{v}_t}{1 - \beta_2^t} \tag{6.3}$$

The parameter update is:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \cdot \frac{\hat{\boldsymbol{m}}_t}{\sqrt{\hat{\boldsymbol{v}}_t} + \epsilon} \tag{6.4}$$

Default hyperparameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-7}$.

6.1.2 Binary Cross-Entropy Loss

For binary classification with sigmoid output $\hat{y} \in (0, 1)$ and ground truth $y \in \{0, 1\}$:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \tag{6.5}$$

This loss is derived from the negative log-likelihood of a Bernoulli distribution. It penalizes confident wrong predictions heavily (due to the log) while being relatively lenient on uncertain predictions. The sigmoid activation ensures $\hat{y} \in (0, 1)$, which is interpreted as $P(\text{spoof} \mid \boldsymbol{x})$.

6.1.3 Callbacks and Training Strategy

Early stopping monitors validation loss and halts training when no improvement is observed for 7 consecutive epochs, then restores the weights from the best epoch. This prevents overfitting without requiring manual monitoring.

ReduceLROnPlateau halves the learning rate after 3 epochs of stagnation. This implements a simple annealing schedule: the optimizer initially takes large steps to find a good basin, then takes smaller steps to refine within that basin.

6.2 Data Generator

The `ASVDataset` class implements a Keras `Sequence` that performs loading, preprocessing, and feature extraction **on the fly** during training. This avoids storing all extracted features in memory simultaneously.

For 2D feature inputs (MFCC, LFCC, CQCC), batch shape is $(B, N_{\text{frames}}, N_{\text{coeffs}}, 1)$. For raw/SincConv inputs, batch shape is $(B, N_{\text{samples}}, 1)$. Training data is shuffled at each epoch end.

6.3 EER Computation

EER is computed from the Receiver Operating Characteristic (ROC) curve:

1. Compute FAR and FRR (equivalently, FPR and $1 - \text{TPR}$) at all unique thresholds.
2. Find the threshold τ^* minimizing $|\text{FAR}(\tau) - \text{FRR}(\tau)|$.
3. Report EER $= \frac{\text{FAR}(\tau^*)+\text{FRR}(\tau^*)}{2} \times 100\%$.

In our implementation, `sklearn.metrics.roc_curve` provides the FPR/TPR pairs, from which FNR $= 1 - \text{TPR}$ gives the FRR.

6.4 Simplified min-tDCF Computation

Our min-tDCF is a **CM-only normalized** version (no tandem ASV system):

$$t\text{-DCF}(\tau) = C_{\text{miss}} \cdot P_{\text{spoof}} \cdot P_{\text{miss}}(\tau) + C_{\text{fa}} \cdot (1 - P_{\text{spoof}}) \cdot P_{\text{fa}}(\tau) \tag{6.6}$$

where $P_{\text{miss}}(\tau) = \frac{|\{i : s_i < \tau, y_i = 0\}|}{|\{i : y_i = 0\}|}$ (fraction of bonafide samples incorrectly rejected) and $P_{\text{fa}}(\tau) = \frac{|\{i : s_i \geq \tau, y_i = 1\}|}{|\{i : y_i = 1\}|}$ (fraction of spoof samples incorrectly accepted).

The minimum over all thresholds is normalized by the cost of a trivial all-accept system:

$$\text{min-tDCF} = \frac{\min_\tau \; t\text{-DCF}(\tau)}{C_{\text{fa}} \cdot (1 - P_{\text{spoof}})} \tag{6.7}$$

With $P_{\text{spoof}} = 0.05$, $C_{\text{miss}} = 1$, $C_{\text{fa}} = 10$.

7 Results

7.1 Summary

Of the 13 planned experiments, **11 completed successfully**. Experiments `cqcc_res2net` and `cqcc_lcnn` were dropped due to dimensional incompatibility between the CQCC output and the Res2Net/LCNN input expectations.

7.2 Comparison Table

Table 7.1: Results of all experiments

Experiment	Feature	Model	Dev EER%	Eval EER%	Dev tDCF	Eval tDCF
mfcc_cnn	MFCC	CNN	8.09	23.87	0.0081	0.0055
mfcc_res2net	MFCC	Res2Net	7.93	23.21	0.0352	0.0247
mfcc_lcnn	MFCC	LCNN	4.09	23.60	0.0076	0.0056
lfcc_cnn	LFCC	CNN	1.08	18.63	0.2484	0.1143
lfcc_res2net	LFCC	Res2Net	0.90	20.98	0.0057	0.0088
lfcc_lcnn	LFCC	LCNN	**0.50**	**16.39**	0.1031	0.0311
cqcc_cnn	CQCC	CNN	9.34	26.82	0.0166	0.0238
cqcc_res2net	CQCC	Res2Net	*Dropped (dimension mismatch)*			
cqcc_lcnn	CQCC	LCNN	*Dropped (dimension mismatch)*			
sincconv_cnn	Sinc	CNN	13.29	33.08	0.0054	0.0053
sincconv_res2net	Sinc	Res2Net	17.03	34.12	0.0054	0.0053
sincconv_lcnn	Sinc	LCNN	9.14	29.45	0.0073	0.0069
raw_rawnet2	Raw	RawNet2	9.53	27.86	0.0053	0.0053

7.3 Best System

Best Combination by Eval EER

- **Experiment:** lfcc_lcnn
- **Feature:** LFCC **Model:** LCNN
- **Eval EER:** 16.39% **Dev EER:** 0.50%
- **Eval min-tDCF:** 0.0311 **Dev min-tDCF:** 0.1031

Note

The large gap between Dev EER and Eval EER (over 30×) indicates severe **overfitting** to the development set. This pattern is consistent across all experiments and is primarily caused by:

- **Unseen attack types:** The evaluation set contains 13 attack algorithms (A07–A19) absent from training, while dev shares the same 6 attacks (A01–A06) as training.
- **Limited training duration:** Models trained for only 8 effective epochs, insufficient for robust feature learning.

7.4 Comparison Chart

Figure 7.1 compares the Eval EER of all successful experiments.

7.5 Training Curves

Training loss and accuracy curves for each experiment are shown in Figures 7.2–7.6.

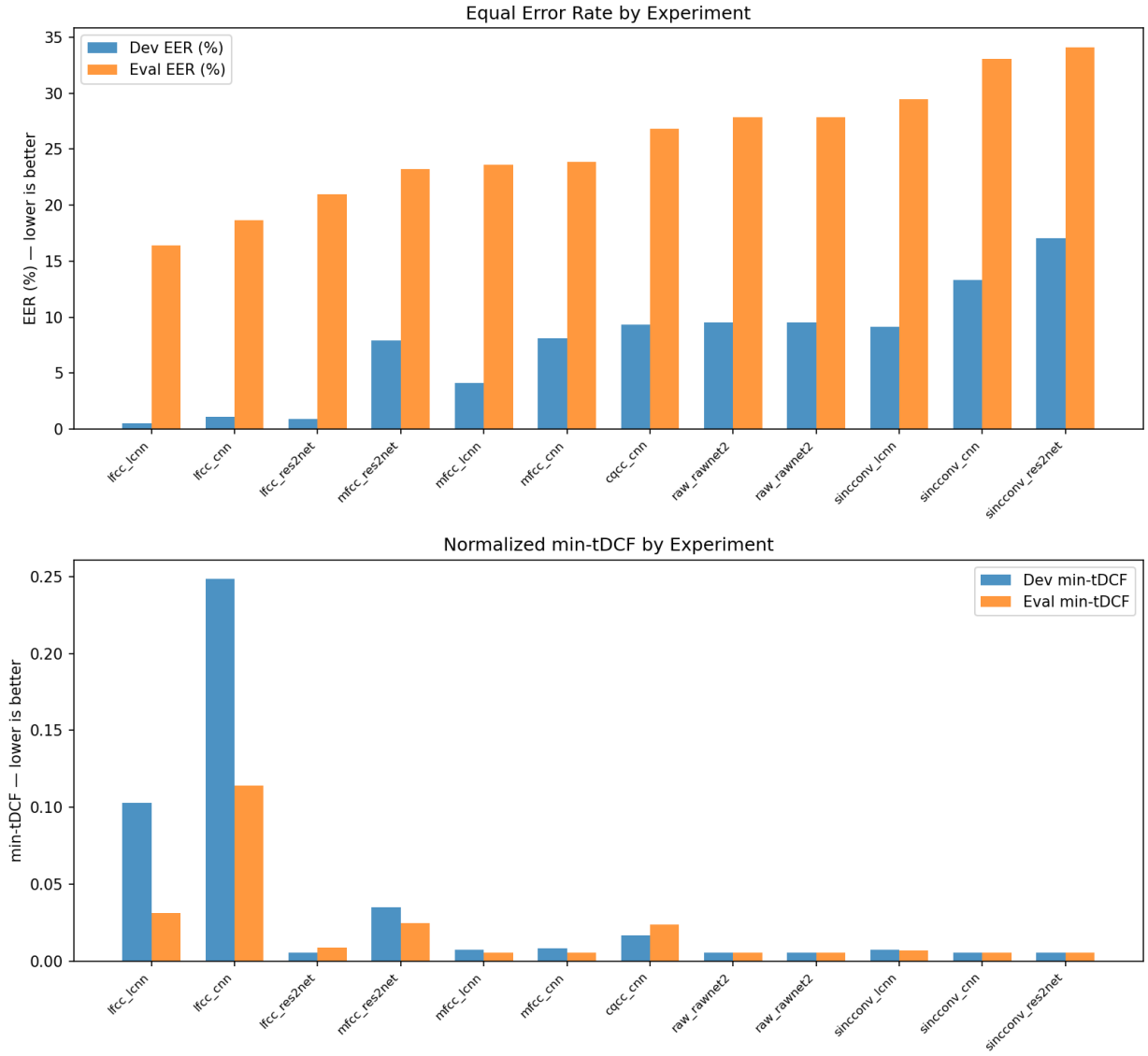

Figure 7.1: Eval EER comparison across experiments — lower is better

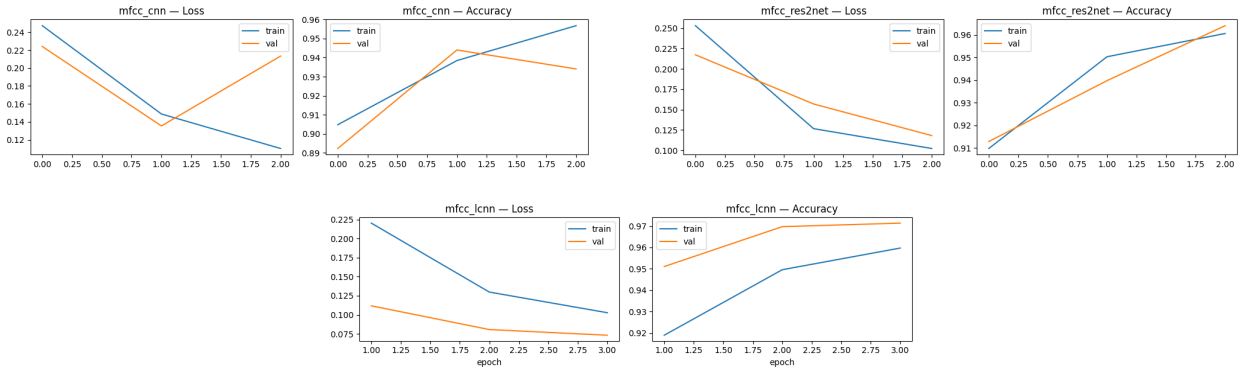

Figure 7.2: Training curves for MFCC combinations

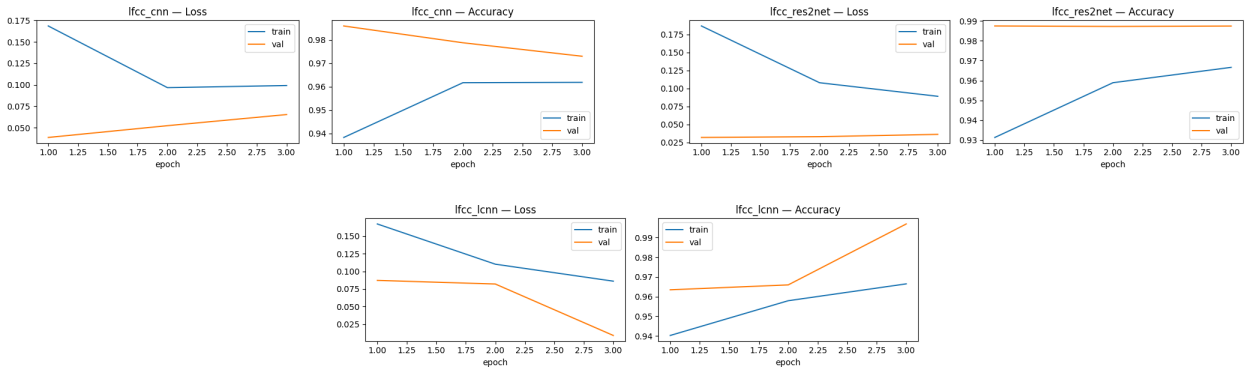

Figure 7.3: Training curves for LFCC combinations

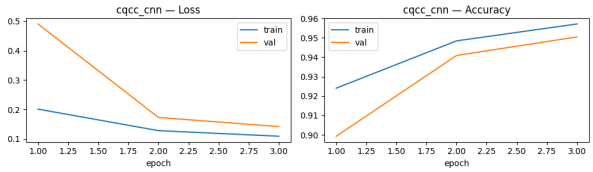

Figure 7.4: Training curve for CQCC+CNN

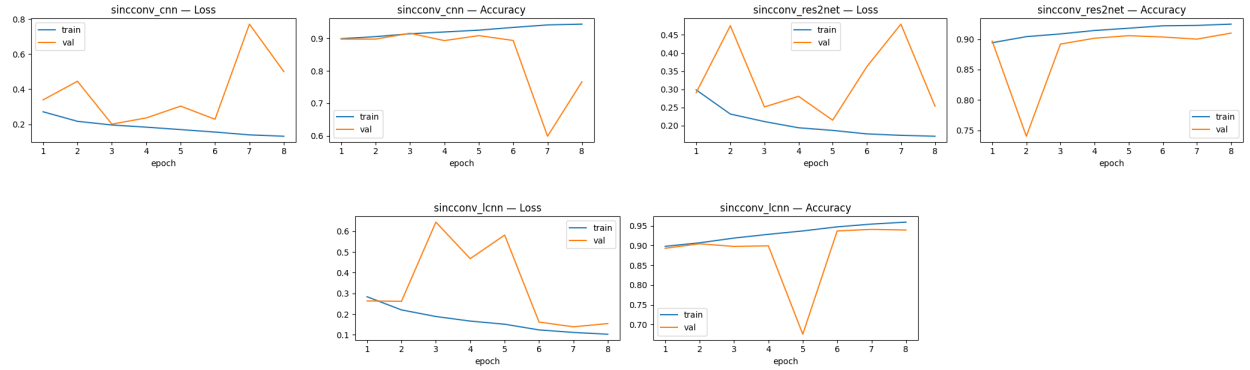

Figure 7.5: Training curves for SincConv combinations

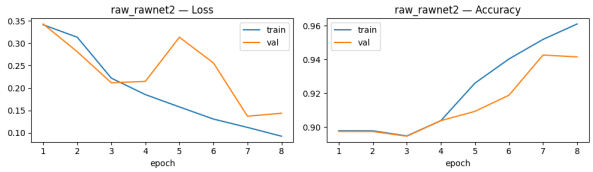

Figure 7.6: Training curve for RawNet2

8 Discussion and Conclusion

8.1 Feature Analysis

Table 8.1: Average Eval EER by feature extractor

Feature	Mean Eval EER%	Experiments
LFCC	18.67	3
MFCC	23.56	3
CQCC	26.82	1
Raw	27.86	1
SincConv	32.22	3

LFCC achieves the best average performance across all feature extractors. The linear filterbank preserves high-frequency information that is crucial for detecting synthesis artifacts. As discussed in Section 4.2, vocoders and TTS systems introduce spectral irregularities above 4 kHz that mel-scaled features (MFCC) attenuate.

MFCC ranks second. Despite its high-frequency attenuation, MFCC still captures sufficient spectral envelope information for reasonable discrimination, particularly for attack types that introduce artifacts across the full spectrum.

CQCC, tested only with CNN (the other two combinations failed), performs worse than both LFCC and MFCC. While CQT's variable resolution is theoretically appealing, the single successful experiment is insufficient to draw strong conclusions. The resampling step required for uniform-time DCT may also introduce interpolation artifacts.

SincConv performs worst. The learnable filterbank requires substantial training to optimize its filter boundaries. With only 8 effective training epochs, the filters likely did not converge far from their mel-scale initialization, resulting in a suboptimal configuration. Longer training schedules (50+ epochs) have been shown to yield significantly better results with SincConv-based models in the literature.

8.2 Architecture Analysis

Table 8.2: Average Eval EER by architecture (shared features only)

Architecture	Mean Eval EER%	Notes
LCNN	23.14	Best average performance
CNN	25.19	Simplest architecture
Res2Net	26.11	Most complex architecture
RawNet2	27.86	End-to-end only

LCNN with MFM activation achieves the best average results. The competitive feature selection mechanism of MFM (Section 5.3) provides an implicit form of feature selection that helps the network focus on discriminative spectral patterns. The lightweight nature of LCNN also means fewer parameters to train, which is advantageous with limited training epochs.

Vanilla CNN is surprisingly competitive, ranking second despite having no attention mechanisms or residual connections. This suggests that with sufficient depth (four blocks) and standard regularization (BatchNorm, Dropout), a simple CNN can extract adequate discriminative features from spectrograms.

Res2Net does not improve over CNN. Despite its multi-scale residual connections and SE attention, Res2Net's additional complexity does not translate to better performance under our training conditions. The multi-scale mechanism requires more training to realize its benefits—with 8 epochs, the hierarchical feature cascades may not have converged.

RawNet2 underperforms feature-based models. End-to-end learning from raw waveforms demands more data and longer training to learn effective representations from scratch, compared to systems that start from well-designed handcrafted features.

8.3 The Generalization Gap

The most striking observation across all experiments is the **massive gap between Dev and Eval EER**. Even the best system (LFCC+LCNN) jumps from 0.50% Dev EER to 16.39% Eval EER—a 33× degradation.

This gap has three primary causes:

1. **Attack mismatch:** Dev and train share the same 6 attack types; the eval set introduces 13 entirely new algorithms, many based on more advanced neural synthesis.
2. **Insufficient training:** 8 epochs is far below the 30–100 epochs typically used in ASVspoof submissions. The networks are undertrained and have not learned generalizable spoofing artifacts.
3. **Limited augmentation:** Only Gaussian/impulsive noise (RawBoost) is applied. More diverse augmentation (SpecAugment, codec simulation, real RIRs) would expose the model to greater input variability during training.

8.4 Conclusion

Proposition 8.1 ▷ Overall Findings

From 11 successful experiments across 4 feature extractors and 4 architectures, three main conclusions emerge:

1. **LFCC is the best feature extractor:** Its linear filterbank preserves high-frequency information where synthesis artifacts are most prominent, yielding consistently lower EER than mel-scaled (MFCC) or log-scaled (CQCC) alternatives.
2. **LCNN with MFM is the most effective architecture:** The Max-Feature-Map activation provides implicit feature selection that outperforms both standard ReLU-based CNNs and more complex Res2Net models under constrained training.
3. **Generalization remains the core challenge:** The 30×+ gap between Dev and Eval EER underscores that detecting *known* attacks is far easier than generalizing to *unseen* synthesis methods. Addressing this requires longer training, richer augmentation, and potentially architecture-level changes (e.g., self-supervised pre-training, graph-based models like AASIST).

Recommendations for Improvement

- Increase training epochs to at least 30 with active early stopping.
- Apply diverse augmentation: SpecAugment, real Room Impulse Responses, codec simulation.
- Explore feature fusion (score-level or embedding-level) to combine complementary

strengths of different features.

- Fix the CQCC dimensional mismatch to enable Res2Net and LCNN experiments.
- Investigate self-supervised learning (SSL) front-ends (wav2vec 2.0, WavLM) which have shown the best generalization in recent ASVspoof editions.
- Consider graph-based architectures (AASIST, RawGAT-ST) that model spectro-temporal dependencies beyond what CNNs can capture.

A Mathematical Foundations

This appendix provides the mathematical background for the signal processing and machine learning concepts used throughout this report.

A.1 The Discrete Fourier Transform (DFT)

The DFT converts a finite-length discrete-time signal $x[n]$, $n = 0, 1, \ldots, N-1$, into its frequency-domain representation:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N}, \quad k = 0, 1, \ldots, N-1 \tag{A.1}$$

The inverse DFT recovers the time-domain signal:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k]\, e^{j2\pi kn/N} \tag{A.2}$$

Each frequency bin k corresponds to the frequency $f_k = k \cdot f_s / N$ Hz, where f_s is the sampling rate. The frequency resolution is $\Delta f = f_s / N$.

A.1.1 Properties of the DFT

- **Linearity:** $\mathrm{DFT}\{a \cdot x + b \cdot y\} = a \cdot X + b \cdot Y$
- **Parseval's theorem:** Total energy is preserved: $\sum_n |x[n]|^2 = \frac{1}{N} \sum_k |X[k]|^2$
- **Conjugate symmetry:** For real-valued $x[n]$: $X[N-k] = X^*[k]$, so only the first $N/2 + 1$ bins carry unique information.
- **Circular convolution theorem:** Multiplication in frequency domain equals circular convolution in time domain: $\mathrm{DFT}\{x \circledast y\} = X \cdot Y$.

A.1.2 The Fast Fourier Transform (FFT)

The FFT (Cooley & Tukey, 1965) is an algorithm that computes the DFT in $O(N \log N)$ operations instead of the naive $O(N^2)$. It exploits the symmetry and periodicity of the complex exponential $W_N^{kn} = e^{-j2\pi kn/N}$ by recursively decomposing the N-point DFT into two $N/2$-point DFTs:

$$X[k] = \underbrace{\sum_{m=0}^{N/2-1} x[2m]\, W_{N/2}^{mk}}_{\text{DFT of even samples}} + W_N^k \underbrace{\sum_{m=0}^{N/2-1} x[2m+1]\, W_{N/2}^{mk}}_{\text{DFT of odd samples}} \tag{A.3}$$

This decomposition is applied recursively until the base case of 2-point DFTs, yielding $\log_2 N$ levels with $O(N)$ operations each.

For our $N_{\text{FFT}} = 512$: naive DFT would require $512^2 = 262\,144$ complex multiplications; FFT requires only $512 \times 9 \approx 4\,608$ — a 57× speedup.

A.2 The Discrete Cosine Transform (DCT)

The DCT is a real-valued transform closely related to the DFT. The **Type-II DCT** (the variant used in MFCC/LFCC/CQCC computation) transforms a sequence $x[n]$ of length N:

$$C[k] = \sum_{n=0}^{N-1} x[n] \cos\left[\frac{\pi k(2n+1)}{2N}\right], \quad k = 0, 1, \ldots, N-1 \tag{A.4}$$

With orthonormal normalization (used in our implementation):

$$C[k] = w_k \sum_{n=0}^{N-1} x[n] \cos\left[\frac{\pi k(2n+1)}{2N}\right], \quad w_k = \begin{cases} \sqrt{1/N} & k = 0 \\ \sqrt{2/N} & k > 0 \end{cases} \tag{A.5}$$

A.2.1 Why DCT for Cepstral Coefficients?

The DCT is used in the final stage of MFCC/LFCC/CQCC extraction for three reasons:

1. **Decorrelation:** Mel/linear filterbank outputs are correlated because adjacent triangular filters overlap. The DCT is the **optimal decorrelating transform** for signals whose autocorrelation matrix is Toeplitz (a good approximation for filterbank energies).

Decorrelated features reduce redundancy and improve the conditioning of downstream classifiers.

2. **Energy compaction:** The DCT concentrates most of the signal energy into the first few coefficients. For the log-filterbank spectrum, the low-order DCT coefficients capture the broad spectral envelope (vocal tract shape), while high-order coefficients capture fine spectral details (pitch harmonics). This natural separation allows truncation to $N_{\text{MFCC}} \ll M$ coefficients with minimal information loss.

3. **Cepstral domain:** The DCT of the log-spectrum is the *cepstrum* (an anagram of "spectrum"). In the cepstral domain, the convolutive effects of the source-filter model (excitation convolved with vocal tract impulse response) become *additive*, making them easier to separate. Low-order cepstral coefficients represent the filter (vocal tract), while high-order ones represent the source (glottal excitation).

A.3 The Mel Scale

The mel scale (Stevens et al., 1937) is a **perceptual scale of pitch** that maps physical frequency (Hz) to perceived pitch. It is approximately linear below 1 kHz and logarithmic above.

$$m = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right) \tag{A.6}$$

$$f = 700 \cdot \left(10^{m/2595} - 1\right) \tag{A.7}$$

A.3.1 Psychoacoustic Basis

The mel scale approximates the **critical band** structure of the human cochlea. The basilar membrane in the inner ear resonates at different positions for different frequencies, with:

- High spatial resolution (narrow critical bands) at low frequencies — this is why we can distinguish tones 100 Hz apart below 1 kHz but cannot distinguish tones 100 Hz apart at 8 kHz.
- Progressively wider critical bands at higher frequencies — the cochlea has approximately 24 critical bands (Barks) spanning 20 Hz to 20 kHz.

A mel-spaced filterbank mirrors this structure: narrow filters at low frequencies (high reso-

lution where human perception is keen) and wide filters at high frequencies (low resolution where perception is coarse). This is optimal for speech recognition (matching human perception) but suboptimal for spoofing detection (where high-frequency artifacts carry discriminative information).

A.4 The Sinc Function and Ideal Filters

The **sinc function** arises naturally as the impulse response of an ideal lowpass filter:

$$\operatorname{sinc}(x) = \frac{\sin(\pi x)}{\pi x}, \quad \operatorname{sinc}(0) = 1 \tag{A.8}$$

An ideal lowpass filter with cutoff f_c (normalized to the sampling rate) passes all frequencies below f_c and blocks all frequencies above. Its impulse response is:

$$h[n] = 2f_c \cdot \operatorname{sinc}(2f_c n) = \frac{\sin(2\pi f_c n)}{\pi n} \tag{A.9}$$

A.4.1 Properties

- The sinc function has **infinite support**: it extends to $\pm\infty$ and decays as $1/n$. In practice, it must be truncated (windowed) to a finite length.
- It oscillates with decreasing amplitude, and its zeros occur at integer multiples of $1/(2f_c)$.
- The frequency response of the truncated (windowed) sinc is no longer an ideal rectangle but has **ripples** (Gibbs phenomenon). The window function controls the tradeoff between main-lobe width (transition bandwidth) and side-lobe level (stopband attenuation).

A.4.2 Bandpass Construction

A bandpass filter passing $[f_1, f_2]$ is constructed as the difference of two lowpass filters:

$$h_{\mathrm{BP}}[n] = h_{\mathrm{LP}}(f_2)[n] - h_{\mathrm{LP}}(f_1)[n] = 2f_2 \operatorname{sinc}(2f_2 n) - 2f_1 \operatorname{sinc}(2f_1 n) \tag{A.10}$$

In the frequency domain, this corresponds to $H_{\mathrm{BP}}(f) = H_{\mathrm{LP}}(f_2) - H_{\mathrm{LP}}(f_1)$, which equals 1 for $f_1 \leq |f| \leq f_2$ and 0 elsewhere.

A.5 Window Functions

Window functions taper the edges of a finite-length signal segment to reduce spectral leakage.

A.5.1 Rectangular Window

The simplest window (equivalent to no windowing):

$$w[n] = 1, \quad 0 \leq n \leq N-1 \tag{A.11}$$

Produces the narrowest main lobe (best frequency resolution) but the highest side lobes ($-13\,\text{dB}$), causing severe spectral leakage.

A.5.2 Hann Window

$$w[n] = 0.5 - 0.5\cos\left(\frac{2\pi n}{N-1}\right) \tag{A.12}$$

Reduces side lobes to $-31\,\text{dB}$ at the cost of a wider main lobe. Commonly used in STFT computations for speech processing.

A.5.3 Hamming Window

$$w[n] = 0.54 - 0.46\cos\left(\frac{2\pi n}{N-1}\right) \tag{A.13}$$

The Hamming window is a modified Hann window that does *not* reach zero at the endpoints. This reduces the nearest side lobe to $-43\,\text{dB}$ (compared to Hann's $-31\,\text{dB}$), providing better stopband attenuation. It is used in our SincConv implementation to window the truncated sinc kernels.

A.5.4 Tradeoff

All window functions present a fundamental tradeoff between:

- **Main-lobe width** (frequency resolution): narrower is better for resolving closely-spaced frequency components.
- **Side-lobe level** (spectral leakage): lower is better for detecting weak components near strong ones.

No window can simultaneously optimize both—this is a consequence of the uncertainty principle for discrete signals.

A.6 The Convolution Theorem

The **convolution theorem** states that convolution in one domain corresponds to point-wise multiplication in the other:

$$x[n] * h[n] \overset{\text{DFT}}{\longleftrightarrow} X[k] \cdot H[k] \tag{A.14}$$

$$x[n] \cdot h[n] \overset{\text{DFT}}{\longleftrightarrow} \frac{1}{N} X[k] * H[k] \tag{A.15}$$

This theorem is foundational to:

- **Filterbank design:** Applying a filter in the time domain (convolution with its impulse response) is equivalent to multiplying in the frequency domain.
- **STFT interpretation:** The STFT can be viewed as applying a bank of frequency-shifted analysis filters to the signal.
- **Source-filter model:** Speech is modeled as the convolution of a glottal excitation source with the vocal tract filter. In the log-spectral (cepstral) domain, this convolution becomes addition, enabling separation of source and filter characteristics.

Proposition A.1 ▷ Source-Filter Separation in the Cepstral Domain

If speech $s[n] = e[n] * v[n]$ (excitation e convolved with vocal tract v), then:

$$|S(f)|^2 = |E(f)|^2 \cdot |V(f)|^2 \tag{A.16}$$

$$\log |S(f)|^2 = \log |E(f)|^2 + \log |V(f)|^2 \tag{A.17}$$

$$\text{DCT}\{\log |S|^2\} = \text{DCT}\{\log |E|^2\} + \text{DCT}\{\log |V|^2\} \tag{A.18}$$

Low-order cepstral coefficients correspond to the slowly-varying vocal tract envelope $V(f)$, while high-order coefficients capture the rapidly-varying excitation harmonics $E(f)$. This separation is why cepstral features are so effective for speech analysis: they naturally disentangle the two fundamental components of speech production.
For spoofing detection, this is relevant because synthetic speech often has unnatural

excitation patterns (e.g., perfectly periodic F0, absence of micro-perturbations) that manifest in the high-order cepstral coefficients.